Jesień Linuksowa

Open Source pod kontrolą

Damian Fajfer

Jesień Linuksowa · Kraków, 23–25.10.2026



O mnie

Damian Fajfer

Architekt Bezpieczeństwa Aplikacji
Bank Pekao S.A.



Bezpieczeństwo SDLC, Platform Engineering, Chmury, Kubernetes

AFILIACJE

- koordynator we **Free Software Foundation Europe**
- Członek **OpenChain Meridian 22**
- Polska Grupa Użytkowników Linuxa
- Warszawski Hackerspace
- Fundacja Wolnego Oprogramowania Reszka

Piątek, 10 grudnia 2021

„Czy mamy Log4j?”

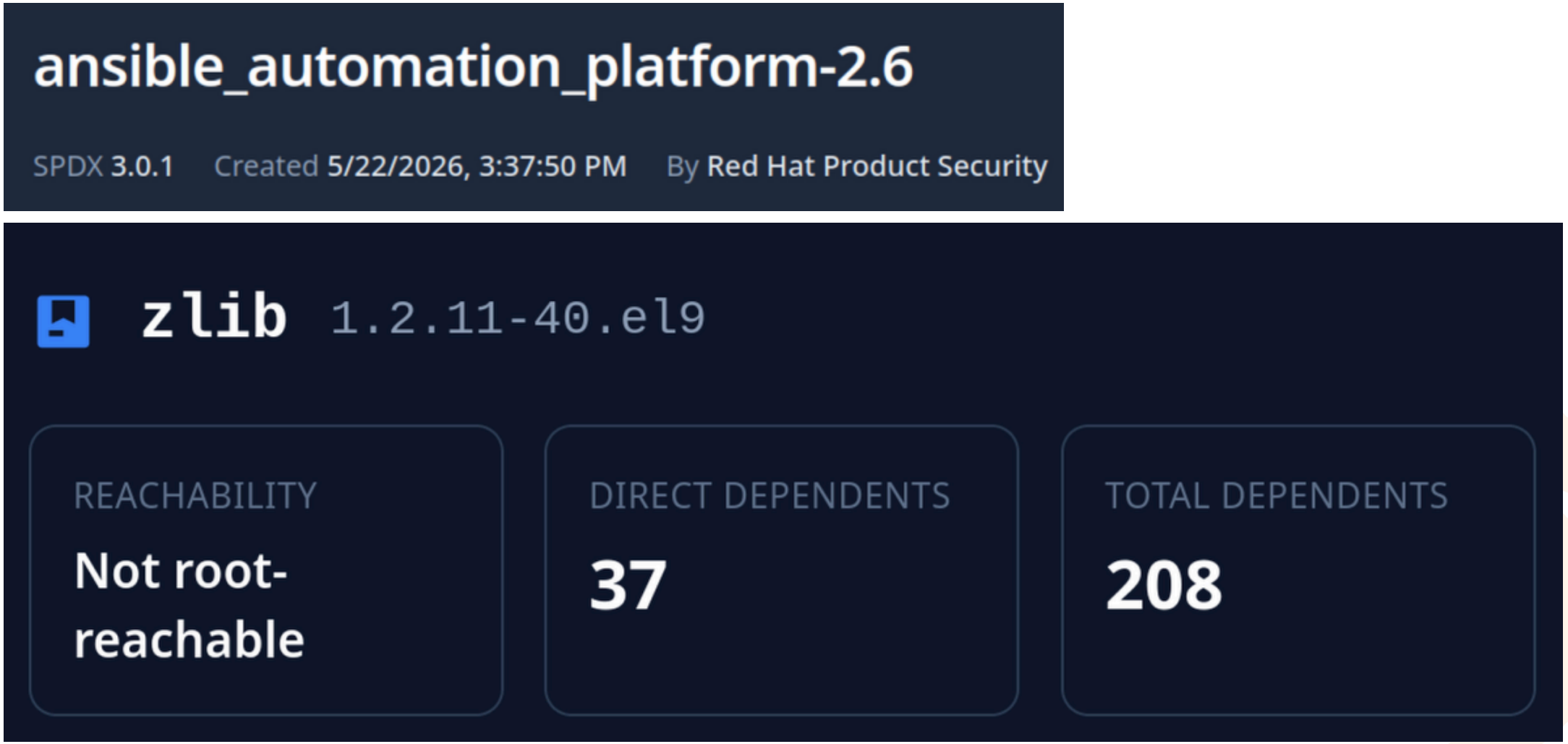
Piątek, 10 grudnia 2021

„Czy mamy Log4j? A jeśli tak, to gdzie?”

Piątek, 10 grudnia 2021

„Czy mamy Log4j? A jeśli tak, to gdzie?”

Kto znał odpowiedź po godzinie? Po dniu? Po tygodniu? Miesiącach?



Źródło: https://tools.spdx.org/app/spdx3_viz/

Wstęp


Tomasz Dedek @tomaszdedek · 1 d
Kolejny incydent i co powiecie SBOM czy wypowiedzenie?
31
7
68

5 lat później

Regulacje wymagają zarządzania otwartymi źródłami

- Wymogi DORA w sektorze finansowym (również dla dostawców)
- Nadchodzące wymogi CRA (przekazywanie SBOMów produktów)

Ktoś mógłby zastanawiać się - po co?

Open Source nie jest dostawcą

Odpowiedzialność jest po stronie użytkownika

1. Jak działa ekosystem Open Source?
2. Mechanizmy warstwowego modelu ochrony
3. Odpowiedzialność po stronie organizacji



CZĘŚĆ 1

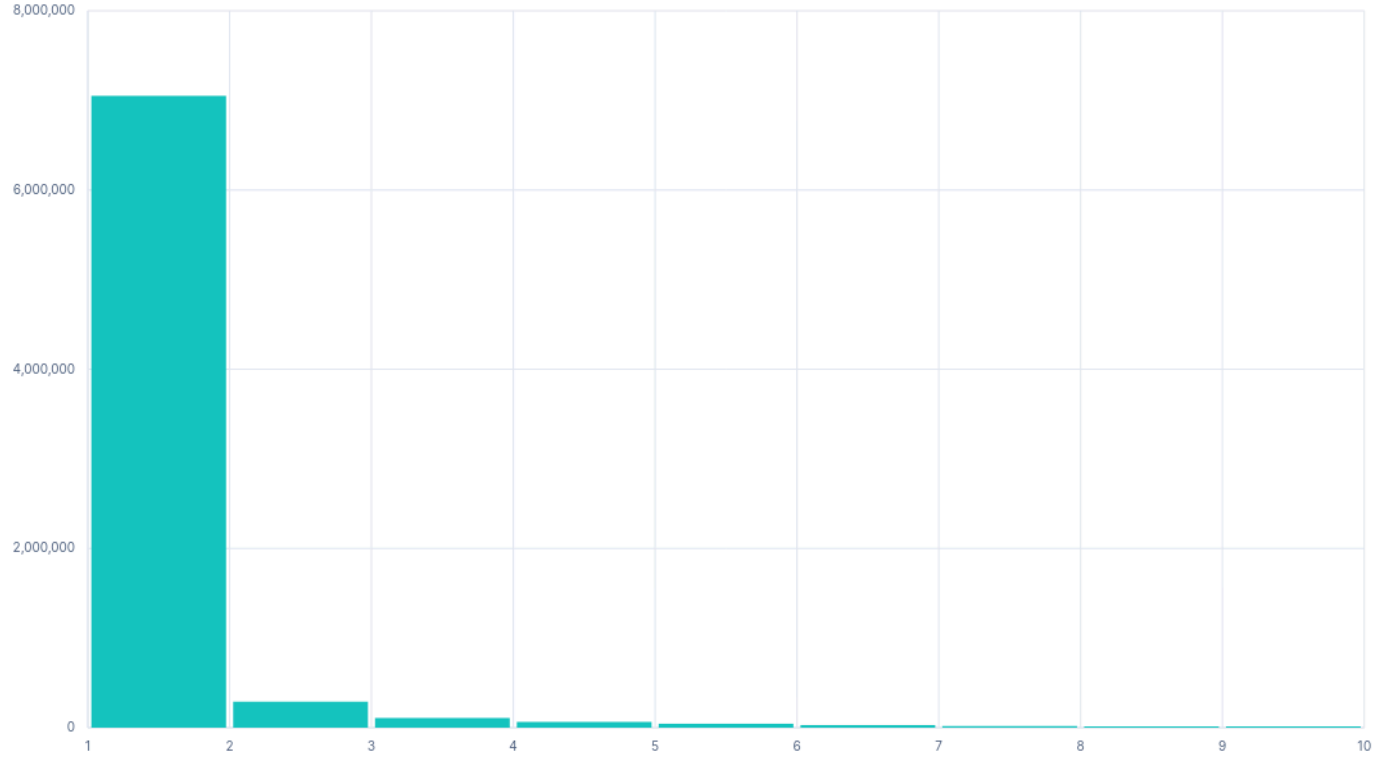
Jak działa ekosystem Open Source

JESIEŃ LINUKSOWA 2026 · KRAKÓW

Na czym stoi Twoja aplikacja?

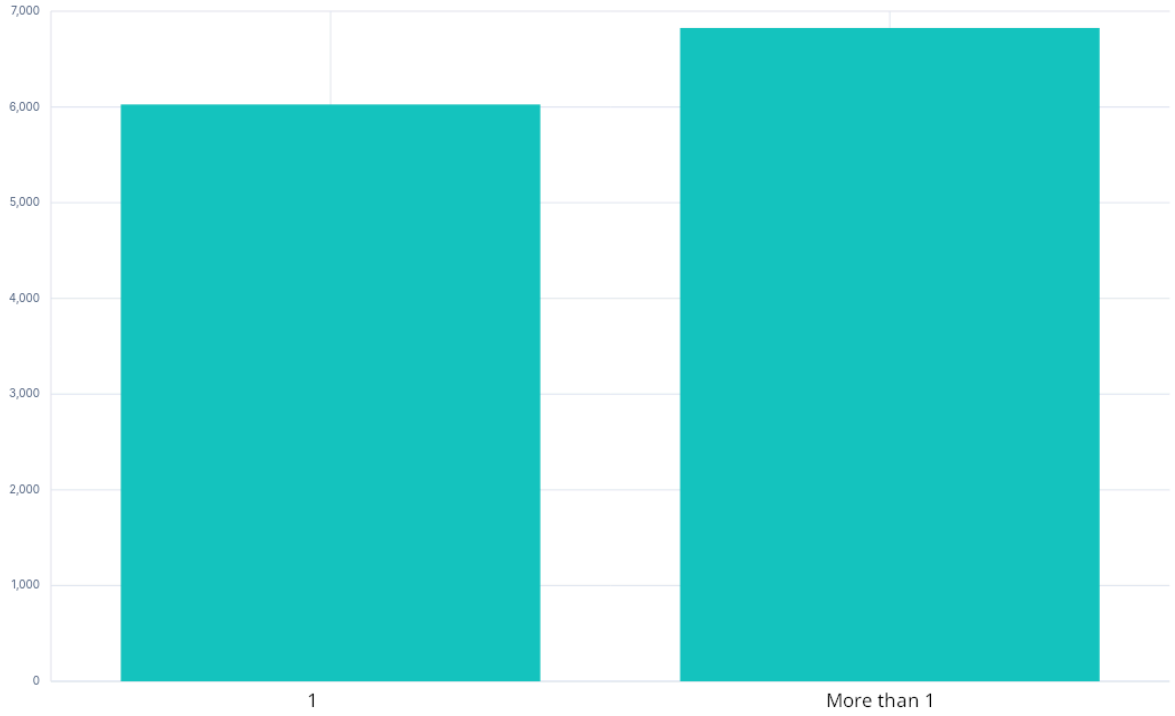
Warstwa	Kto ją utrzymuje?
Twoja aplikacja	Ty
Framework	fundacje, korporacje, konsorcja
Biblioteka	społeczność, od jednej do kilku osób
Zależności Twoich zależności	zwykle nikt nie sprawdza
Inna biblioteka	**jedna osoba, po godzinach**
Obraz bazowy, system, środowisko uruchomieniowe	fundacje, korporacje, konsorcja

Większość oprogramowania ma jednego konserwatora



Źródło: https://opensourcesecurity.io/2025/08-oss-one-person/

Projekty powyżej 1 mln pobrań miesięcznie



Źródło: https://opensourcesecurity.io/2025/08-oss-one-person/

Mapa ekosystemu

FUNDACJE

Apache, Linux Foundation, Eclipse, PSF. Zarządzanie, infrastruktura, standardy.

FIRMY, KONSORCJA

Zatrudniają maintainerów, sprzedają wsparcie i produkty z open source w środku.

MAINTAINERZY

Często wolontariusze. Nie mają wobec Ciebie żadnych zobowiązań (eg. Daniel Stenberg i korporacja MetLife)

DYSTRYBUCJE

Debian, Fedora, Rancher. Wybierają, budują, łatają i utrzymują stabilne wydania.

REJESTRY

npm, PyPI, Maven Central, Docker Hub. Hostują paczki, nie gwarantują ich jakości.

TY

Bierzesz to wszystko i wdrażasz na produkcję.

Droga paczki: od zmiany do produkcji

	Źródło	Build	Rejestr	Wejście	Nasz build	Środowisko
Kto/co	maintainer	CI projektu	npm, PyPI, Maven Central	proxy, mirror	nasz CI, środowiska deweloperskie	produkcja
Zakładamy, że…	kod jest bezpieczny	artefakt powstał z tego kodu	pobieramy ten artefakt	to jest dokładnie ten sam artefakt	budujemy ze sprawdzonych składników	uruchamiamy to, co zbudowaliśmy

Maintainer nie jest Twoim dostawcą

> THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND…
>
> – licencja MIT

NIE MA

- umowy, SLA ani terminu na poprawkę
- obowiązku odpowiadania na zgłoszenia i maile
- gwarancji, że projekt będzie przez niego utrzymywany

JEST

- kod, który możesz użyć, przeanalizować, dzielić się nim i go poprawiać
- licencja, która Ci na to pozwala
- społeczność, do której możesz dołączyć

Dlaczego jest to istotne?

ATAKUJĄCY ZMIENILI CEL

Włamywanie się do firm jest trudniejsze i bardziej kosztowne. Łatwiej zaatakować łańcuch dostaw.

REGULACJE

CRA, NIS2, DORA wymuszają na organizacjach wiedzę o składnikach oprogramowania i zarządzanie ryzykiem

SKRÓCONY CZAS REAKCJI

Coraz więcej CVE i coraz mniej czasu na ocenę. Sztuczna inteligencja przyspiesza rozwój oprogramowania jak i szybciej wykorzystuje luki.

Ekosystem jest wspólny dla nas wszystkich, powyższe problemy dotyczą nas wszystkich

Skrócony czas reakcji

Hi,

https://lists.debian.org/debian-security-an
notes that:

"Several vulnerabilities have been discover
Linux kernel that may lead to a privilege e
denial of service or information leaks."

Where "several" is a list of 1,313 CVE IDs.

Aided in large part by AI-assisted vulnerability discovery, Microsoft's October Patch Tuesday update has once again addressed a record number of common vulnerabilities and exposures (CVEs), with over 1,400 flaws listed, more than in the entirety of 2023.

https://seclists.org/oss-sec/2026/q3/991

https://www.computerweekly.com/news/366652119/Microsoft-CVE-volumes-on-relentless-upward-trajectory



CZĘŚĆ 2

Mechanizmy warstwowego modelu ochrony

Mechanizmy warstwowego modelu ochrony



Mechanizmy warstwowego modelu ochrony



Źródło: https://aniagotuje.pl/przepis/tort-malinowy-z-truskawkami-w-czekoladzie

Mechanizmy warstwowego modelu ochrony



Matryca pytań

Pytanie	Etap łańcucha	Mechanizmy
➔ Co mam?	nasz build, produkcja	SCA, SBOM, VEX
Skąd to przyszło?	build, rejestr	podpisy (Sigstore), SLSA
Czy chcę to wpuścić?	źródło, wejście	Scorecard, proxy, pinning, karencja
Czy wdrażam to, co zbudowano?	nasz build, produkcja	podpis, poświadczenie, weryfikacja przy wdrożeniu

> Gdy nie budujemy sami, tylko wdrażamy istniejące oprogramowanie od dostawców mechanizmy bezpieczeństwa pozostają bez zmian

SCA - Software Composition Analysis

Narzędzie rozpoznaje komponenty oprogramowania i porównuje je z bazami znanych podatności.

> Czy w artefaktach, które rozpoznałem, są znane podatności?

POTWIERDZA

- podatności w dopasowanych komponentach
- według stanu konkretnej bazy, tylko w momencie skanowania

NIE GWARANTUJE

- wykrycia malware ani 0-day
- przeskanowania wszystkich warstw komponentu

KOMENTARZ

- skanuj to, co trafia do organizacji
- podatny kod nie musi być osiągalny

SBOM - Software Bill of Materials

Lista składników oprogramowania w formacie maszynowym (SPDX lub CycloneDX)

> Na co składa się dane oprogramowanie?

POTWIERDZA

- deklarację, co znalazło się w artefakcie
- stan na moment generowania (aktualizacji)

NIE GWARANTUJE

- kompletności ani poprawności
- opisu uruchamianego artefaktu

KOMENTARZ

- SBOM powinien być żywym dokumentem, warto pielęgnować te dane (CBOM, VEX)
- CISA Minimum Elements for SBOM (29.07.2026)

VEX

Log4Shell, podejście drugie

GDZIE MAMY LOG4J? - SBOM

```
# przy każdym buildzie obrazu
syft registry.example.com/app@sha256:… \
  -o cyclonedx-json > sboms/app.cdx.json

# piątek, 18:00
grep -l log4j-core sboms/*.json
```

CZY NAS DOTYCZY? - VEX

```
{
  "vulnerability": { "name": "CVE-2021-44228" },
  "products": [{ "@id": "pkg:oci/app@sha256%3A…" }],
  "status": "not_affected",
  "justification": "vulnerable_code_not_present"
}
```

> **VEX** potwierdza jedynie **deklarację** autora, że podatność go (nie) dotyka.

Matryca pytań

Pytanie	Etap łańcucha	Mechanizmy
~~**Co mam?**~~	~~nasz build, produkcja~~	~~SCA, SBOM, VEX~~
→ Skąd to przyszło?	build, rejestr	podpisy (Sigstore), SLSA
Czy chcę to wpuścić?	źródło, wejście	Scorecard, proxy, pinning, karencja
Czy wdrażam to, co zbudowano?	nasz build, produkcja	podpis, poświadczenie, weryfikacja przy wdrożeniu

Podpis artefaktu (np. Sigstore)

Dowód, kto opublikował artefakt. Stan sztuki, Sigstore, stosuje podpis tożsamością zamiast klucza.

> Kto to opublikował i czy ktoś to po drodze zmienił?

POTWIERDZA

- integralność od chwili podpisu
- tożsamość podpisującego
- publiczny ślad w rejestrze Rekor (z możliwością użycia niezależnego serwera TSA)

NIE GWARANTUJE

- że podpisujący jest godny zaufania
- że jego konto albo CI nie zostało przejęte
- że kod jest bezpieczny

KOMENTARZ

- samo istnienie podpisu samo w sobie nic nie dowodzi
- analogicznie do 'zielonej kłódki'

SLSA

Supply-chain Levels for Software Architects (SLSA) - framework (zestaw standardów i kontrolek bezpieczeństwa) do zabezpieczania łańcucha dostaw

> Czy ciężko jest ingerować w artefakt pomiędzy commitem a wynikiem końcowym procesu?

Poziom	Wymóg	Nacisk na
L1	Utworzony provenance (pochodzenie) artefaktu	Dokumentację
L2	Podpisany provenance w procesie CI	Brak ingerencji po zbudowaniu artefaktu
L3	Środowisko nie pozwalające na zewnętrzną ingerencję w artefakt	Brak ingerencji podczas budowy artefaktu

Provenance

Provenance (pochodzenie): poświadczenie, z czego i gdzie zbudowano artefakt.

> Jak, gdzie i z czego to zbudowano?

POTWIERDZA

- zbudowano na platformie X z repo Y, commit Z
- dla SLSA L3 build nie podrobi poświadczenia

NIE GWARANTUJE

- że podpisujący jest godny zaufania
- że jego konto albo CI nie zostało przejęte
- że zależności są bezpieczny

KOMENTARZ

- wymagaj provenance od własnych buildów

Weryfikacja w praktyce

```
# podpis obrazu: sprawdzamy nie tylko CZY, ale KTO podpisał
cosign verify registry.example.com/app@sha256:… \
  --certificate-identity https://github.com/org/app/.github/workflows/release.yml@refs/heads/main \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com

# provenance (SLSA) z GitHub Artifact Attestations
gh attestation verify oci://registry.example.com/app@sha256:… --repo org/app

# podpisy rejestru i provenance zainstalowanych paczek npm
npm audit signatures
```

Bez weryfikacji konkretnych kluczy weryfikacja potwierdza tylko, że **ktoś** podpisał.

xz-utils (2024) oczami obrońcy

Mechanizm	Pomógłby?
Podpisywanie artefaktów	nie - podpisał je maintainer
SCA	nie - do ujawnienia nie mamy CVE
SBOM	**tak** - po ujawnieniu ataku
Build ze źródeł	**tak** - wyjątkowo, brak backdooru w gicie
Provenance	**częściowo** - dopiero przy SLSA L3 (podpis przez OIDC)
Stabilny kanał dystrybucji	**tak** - stabilne wydania nie zdążyły przyjąć tej wersji
Człowiek	**tak** - wykrył to inżynier, któremu SSH logował się 500ms wolniej

Matryca pytań

Pytanie	Etap łańcucha	Mechanizmy
~~**Co mam?**~~	~~nasz build, produkcja~~	~~SCA, SBOM, VEX~~
~~**Skąd to przyszło?**~~	~~build, rejestr~~	~~podpisy (Sigstore), SLSA~~
➔ Czy chcę to wpuścić?	źródło, wejście	Scorecard, proxy, pinning, karencja
Czy wdrażam to, co zbudowano?	nasz build, produkcja	podpis, poświadczenie, weryfikacja przy wdrożeniu

Metryki CHAOSS



CHAOSS Practictioner Guides jako kontekst do LLMa (prompty, serwery MCP)

OpenSSF Scorecard

Automatyczna ocena praktyk w repozytorium projektu (review, CI, wydania).

> Czy projekt stosuje dobre praktyki?

POTWIERDZA

- heurystyki higieny repozytorium
- stan na dany moment (aktualizacja raz na tydzień / na żądanie)

NIE GWARANTUJE

- bezpieczeństwa kodu
- niski wynik ≠ niebezpieczny
- wysoki wynik ≠ bezpieczny

KOMENTARZ

- OpenSSF Concise Guide for Evaluating Open Source Software

Checklista: https://best.openssf.org/Concise-Guide-for-Evaluating-Open-Source-Software.html

Higiena korzystania z zewnętrznych paczek

Trzy praktyczne mechanizmy

- Proxy/mirror - miejsce składowania zasobów z internetu po stronie organizacji
  - daje nam odporność na niedostępność zasobu sieciowego
  - zapewnia pobranie tego samego artefaktu w całej organizacji
- Pinning - zaczepienie się, na jedną, konkretną instancję artefaktu w danej wersji
- Karencja - czas od powołania nowej paczki, podczas którego nie pobieramy jej

Wejście do organizacji

	Proxy / mirror	Pinning	Karencja
Pytanie	Skąd pobieramy paczki?	Czy instaluję to samo co wczoraj?	Czy wersja żyje kilka dni publicznie?
Potwierdza	jedno wejście, log, możliwość blokady	niezmienność bajt w bajt	czas na wykrycie przez społeczność i skanery
Nie gwarantuje	że to, co przepuszcza, jest dobre	że przypięta wersja była bezpieczna	że ktokolwiek tę wersję przeanalizował
Zalecenia	blokuj znane złośliwe paczki, chroń nazwy wewnętrzne	aktualizuj automatycznie (Renovate, Dependabot)	**szybka ścieżka dla poprawek bezpieczeństwa**

Matryca pytań

Pytanie	Etap łańcucha	Mechanizmy
~~**Co mam?**~~	~~nasz build, produkcja~~	~~SCA, SBOM, VEX~~
~~**Skąd to przyszło?**~~	~~build, rejestr~~	~~podpisy (Sigstore), SLSA~~
~~**Czy chcę to wpuścić?**~~	~~źródło, wejście~~	~~Scorecard, proxy, pinning, karencja~~
→ Czy wdrażam to, co zbudowano?	nasz build, produkcja	podpis, poświadczenie, weryfikacja przy wdrożeniu

Czy wdrażam to, co zbudowano?

- Bądźmi gotowi na ataki z wewnątrz
- Stosujmy najlepsze praktyki (podpisywanie, poświadczanie et al.) w skali i w sposób zautomatyzowany
- Automatyczna weryfikacja silnikami polityk na środowiskach
- Polityki jako kod konsultowane/tworzone z zespołami bezpieczeństwa
- Korzystajmy z efemerycznych workerów, każda akcja w CI powołuje nowy kontener/maszynę
  - cache poisoning
  - potencjalnie wrażliwe dane leżące na maszynach

CI/CD to też łańcuch dostaw

- **Minimalne uprawnienia** tokenów w pipeline'ach
- **Akcje** - lokalne kopie mogą tworzyć dług technologiczny. Warto wymusić na nich analizę statyczną (np. zizmor)
- **Obrazy z przypiętymi SHA** - kolejne zależności, zwykle nieobecne w SBOM aplikacji
- **Efemeryczne workery** - każdy build na czystym środowisku

Żaden mechanizm nie wystarczy sam

Incydent	SBOM + SCA	Podpis	Provenance	Pinning	Karencja
Log4Shell (2021)	■	▫	▫	▫	▫
xz-utils (2024)	■	▫	■	▫	■
Ultralytics (2024)	■	▫	■	■	■
tj-actions (2025)	▫	▫	▫	■	■
chalk/debug, Shai-Hulud (2025)	■	▫	▫	■	■

■ pomaga · ■ częściowo · ▫ nie pomaga

Każda warstwa jest dziurawa. Nie pozwól, żeby dziury ułożyły się w jednej linii.

Warstwy obrony

Warstwa	Co robimy	Mechanizmy
Analiza	analiza, czy można to włączyć	Concise Guide, Scorecard, analiza manualna
Wejście	kontrola, co i kiedy wpada	proxy, pinning, karencja
Build	wiemy, z czego i jak zbudowaliśmy	SBOM, podpis, provenance, utwardzone CI
Wdrożenie	uruchamiamy tylko to, co zbudowaliśmy	weryfikacja przy wdrożeniu
Eksploatacja	trzymamy fabrykę w ryzach	SCA na SBOM-ach, VEX
Ludzie	ktoś za to wszystko odpowiada	OSPO, zespoły analityczne ds. aplikacji



CZĘŚĆ 3

Odpowiedzialność po stronie organizacji

Regulacje: dostawcy ICT, w tym open source

	Kogo dotyczy	Co z tego wynika dla open source
CRA	producentów produktów z elementami cyfrowymi	należyta staranność przy komponentach (także FOSS), SBOM, obsługa podatności
NIS2 / KSC	podmiotów kluczowych i ważnych	bezpieczeństwo łańcucha dostaw, obsługa podatności
DORA	sektora finansowego	ryzyko ICT stron trzecich, śledzenie bibliotek (także open source)

Wspólny mianownik: wiedz, co masz, oceń ryzyko, zarządzaj podatnościami przez cały cykl życia, udokumentuj i monitoruj ten proces

OSPO - Open Source Program Office

Zespół osób w organizacji, których zadania stykają się przy zarządzaniu open sourcem. Dostawca open source w organizacji

- **Strategia i zasady** - jak korzystamy z open source i jak kontrybuujemy
- **Zgodność** - licencje, regulacje, SBOM dla klientów
- **Bezpieczeństwo** - razem z AppSec: dopuszczanie, monitorowanie, wycofywanie
- **Upstream** - kontakt z projektami, kontrybucje, finansowanie
- **Edukacja** w zakresie ekosystemu otwartego oprogramowania

> OSPO nie musi być dedykowanym biurem. Często jest wirtualnym ciałem w organizacji.

Trendy rynkowe

- 79% badanych organizacji planuje stworzyć OSPO w ciągu 2 następnych lat
- Co trzeci badany wskazuje bezpieczeństwo, zgodność, oszczędność w rozwijaniu oprogramowania i zmniejszenie kosztu ponownego użycia jako uzasadnienie biznesowe
- 58% OSPO jest bezpośrednio zaangażowanych w tematykę AI w kontekście ładu, zarządzania, bezpieczeństwa i tworzenia standardów

Źródło: Trinkenreich, B. Santamaría, A.J., Lawson, A. 2026. "The 2026 State of OSPOs and Open Source Management." Research Report. Linux Foundation Research. https://doi.org/10.70828/UCLW3741

Cykl życia komponentu

DOPUSZCZENIE

- uzasadnienie biznesowe
- dostępne frameworki i checklisty jako wsparcie
- licencja zgodna z polityką
- wejście tylko przez proxy

MONITOROWANIE

- SBOM każdego artefaktu, odpytywany codziennie
- aktualizacje automatyczne, z karencją
- szybka ścieżka dla poprawek bezpieczeństwa

WYCOFANIE

- koniec wsparcia, porzucony projekt, zmiana licencji, kompromitacja
- zamiana, fork, płatne wsparcie
- wyjątek tylko na jakiś okres

Mapa dojrzałości dla konsumenta: OpenSSF S2C2F (Secure Supply Chain Consumption Framework)

OpenChain: ISO/IEC 5230 i ISO/IEC 18974

Standardy zarządzania open sourcem w organizacji

ISO/IEC 5230

Zgodność licencyjna: polityka, role, kompetencje, lista komponentów, artefakty zgodności, kontrybucje.

ISO/IEC 18974

Bezpieczeństwo: SBOM, wykrywanie i obsługa znanych podatności, także po wydaniu.

PO CO?

Zaufanie w łańcuchu dostaw.
Tego samego możesz wymagać od dostawców. Dostęp do standardów jest bezkosztowy.

Odpowiedzialność to też wkład własny

- **Stwórz proces** - pozwól ludziom zgłaszać poprawki, kontrybuować do kluczowych dla Twojej organizacji projektów
- **Zgłaszaj i poprawiaj u źródła** - poprawka w lokalnej kopii pomaga tylko Tobie, dodatkowy fork ciąży tylko organizacji
- **Wspieraj ekosystem** - czas inżynierów, pieniądze, członkostwo w organizacjach
- **Dbajmy o naszą wspólną infrastrukturę** - poruszamy się po tych samych drogach, otwarty ekosystem i jego bezpieczeństwo to zmartwienie i wspólne dobro każdego z nas

Open Source nie jest dostawcą

Odpowiedzialność jest po stronie użytkownika

- Organizacja musi bezpieczenie korzystać z oprogramowania otwartoźródłowego
- Model obrony jest warstwowy, wymusza zbudowanie procesów
- Nie kupujcie magicznych narzędzi do zabezpieczania łańcucha dostaw

Jesień Linuksowa

Dziękuję

Q&A

